04

AULA 04 · Segunda, 17/08

Proxy Reverso e DNS

Intermediários inteligentes e o catálogo da internet

Módulo · Arquitetura de Serviços e Dados

Prof.: Everton J Benedicto

ada

Cronograma das Aulas

01
Seg 10/08
Introdução ao Módulo e Docker

02
Qui 13/08
Arquitetura de Servidores

03
Sex 14/08
Apache e Nginx

04
Seg 17/08
Proxy Reverso e DNS

05
Qua 19/08
CI/CD e Ciclo de Vida

06
Sex 21/08
Cloud Computing

07
Seg 24/08
Arquitetura de Dados e Data Warehouse

08
Ter 25/08
Data Lake

09
Qua 26/08
Concluir atividades e preencher rubricas

COMBINADO DA TURMA

Por favor!

Bora participar!



Câmera ligada + mão levantada
= aula top!

INTERVALO · 16:30 BRT

Combinados da nossa aula

1. Câmeras ligadas.
2. Entrem no horário: início e intervalo.
3. Participem da aula.
4. Tirem dúvidas.
5. Levantem a mão para tirar dúvidas.

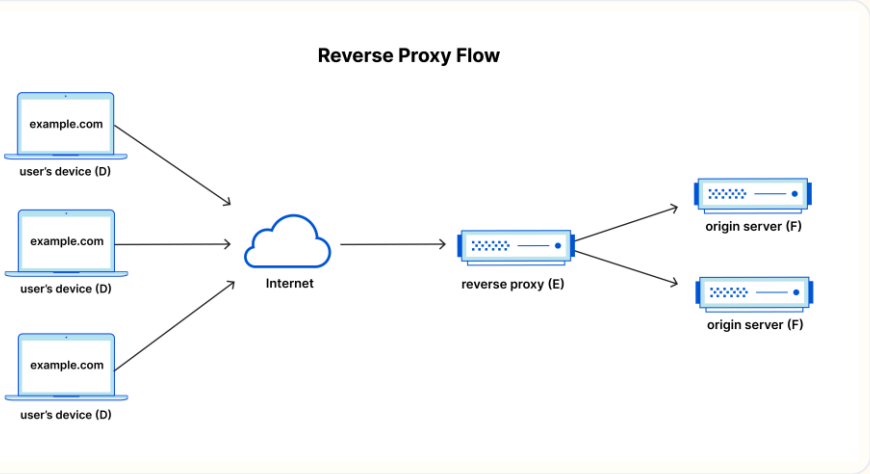
REVIEW!

Recapitulando a Aula 3

- ✓ Apache: veterano e modular, uma thread/processo por requisição — ótimo para flexibilidade.
- ✓ Nginx: assíncrono por eventos — ótimo para alta concorrência e proxy reverso.
- ✓ Podem trabalhar juntos: Nginx na frente (proxy) + Apache/Tomcat atrás (app).
- ✓ Docker facilita instalar e testar qualquer servidor web.

PROXY REVERSO

Proxy Reverso — O Segurança do Prédio



1. **Fica na frente:** recebe as requisições dos clientes
2. **Esconde o backend:** o cliente só fala com o proxy
3. **Balanceia carga:** distribui o tráfego entre servidores
4. **Protege e faz cache:** absorve ataques e acelera respostas

Fonte: cloudflare.com

PROXY REVERSO + DOCKER

Nginx como proxy reverso no Docker

Um container Nginx na frente; as APIs ficam atrás, na rede interna.

Passo a passo

1 · Escreva o nginx.conf com os upstreams e as rotas /api.

2 · Crie a imagem do gateway a partir do nginx:alpine.

3 · Publique apenas a porta 8080 do gateway.

4 · Coloque todos os containers na mesma rede Docker.

estrutura do projeto

```
projeto/
├── docker-compose.yml
├── gateway/
│   ├── Dockerfile
│   └── nginx.conf
├── messages/
└── payments/
```

MÃO NA MASSA

O arquivo nginx.conf

Cada upstream aponta para o nome do container; o location repassa o caminho.

gateway/nginx.conf

```
upstream messages { server messages:3000; }
upstream payments { server payments:4000; }

server {
    listen 8080;

    location /api/messages/ {
        proxy_pass http://messages/;
        proxy_set_header Host $host;
    }

    location /api/payments/ {
        proxy_pass http://payments/;
    }
}
```

REDE

Rede Docker: só o gateway é exposto

Sem portas publicadas, as APIs existem apenas dentro da rede interna.

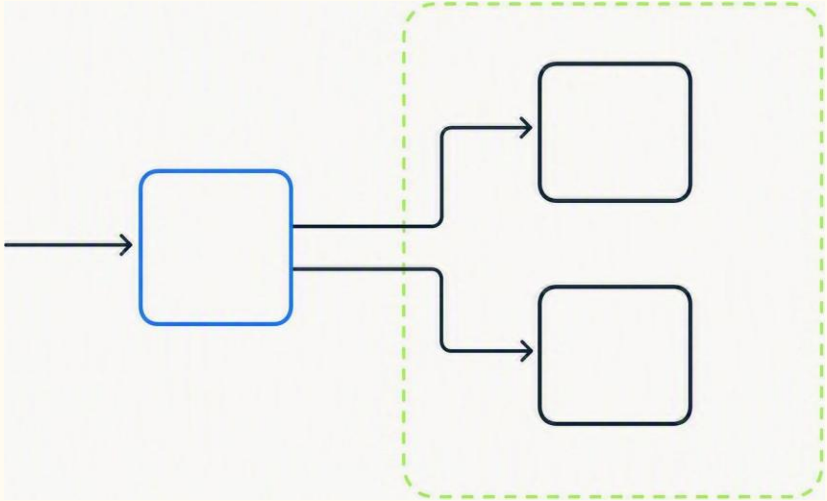
Isolando as APIs

Crie uma rede bridge própria: os containers se enxergam pelo nome do serviço.

Só o gateway publica portas (8080:80) para o host.

As APIs não publicam nada — usam expose e ficam invisíveis de fora.

Resultado: a única porta de entrada é o proxy reverso.



Cliente → gateway (8080) → APIs na rede interna

DOCKER COMPOSE

Toda a topologia em um docker-compose.yml

Um arquivo descreve serviços, rede e dependências da aplicação.

Por que usar Compose

- ✓ **Reprodutível**: o ambiente inteiro sobe com um docker compose up.
- ✓ A **rede** e o **DNS** interno dos serviços são criados automaticamente.
- ✓ **depends_on** organiza a ordem de subida dos containers.
- ✓ **Infra como código**: tudo versionado no Git junto do projeto.

docker-compose.yml

```yaml
services:
  meu-gateway:
    ports: ["8080:80"]
    networks: [rede-publica, rede-interna]
    depends_on: [messages, payments]
  messages:
    build: ./messages
    networks: [rede-interna]
  payments:
    build: ./payments
    networks: [rede-interna]
networks:
  rede-publica:
  rede-interna:
    internal: true
```

MÃO NA MASSA

Proxy Reverso e APIs

Crie um proxy reverso que recebe todas as requisições de duas APIs, na porta 8080/api:

APIs de origem

messages — endpoint day: retorna a mensagem diária.
payments — endpoints pix, creditcard e debit: recebem o valor a ser pago na própria URL.

Containers

meu-gateway · messages · payments

requisições ao gateway

```
GET   http://localhost:8080/api/messages/day
POST  http://localhost:8080/api/payments/pix/100.50
POST  http://localhost:8080/api/payments/creditcard/250.75
POST  http://localhost:8080/api/payments/debit/80.25
```

O gateway escuta em localhost:8080/api e encaminha para cada API.

LABORATÓRIO

Comandos do laboratório

Execute na ordem, a partir da pasta do projeto, no PowerShell.

Windows PowerShell

```
# 1. Construir as imagens e iniciar os containers
PS C:\projeto> docker compose up --build -d

# 2. Conferir o estado dos containers
PS C:\projeto> docker compose ps

# 3. Visualizar os logs, se necessário
PS C:\projeto> docker compose logs –follow meu-gateway messages payments

# 4. Testar a API messages
PS C:\projeto> Invoke-RestMethod http://localhost:8080/api/messages/day

# 5. Testar pagamento via PIX
PS C:\projeto> Invoke-RestMethod -Method Post http://localhost:8080/api/payments/pix/100.50

# 6. Testar pagamento com cartão de crédito
PS C:\projeto> Invoke-RestMethod -Method Post http://localhost:8080/api/payments/creditcard/250.75

# 7. Testar pagamento com débito
PS C:\projeto> Invoke-RestMethod -Method Post http://localhost:8080/api/payments/debit/80.25
```

DNS — O Catálogo Telefônico da Internet

1. **Traduz:** www.google.com em um endereço IP
2. **Sem DNS:** você teria que decorar o IP de cada site!
3. **DNS Autoritativo:** tem a resposta final de um domínio
4. **DNS Recursivo:** busca e guarda a resposta em cache
5. **Na nuvem:** AWS usa Route 53; Azure usa Azure DNS

💡 ANALOGIA

O DNS é a agenda de contatos do celular: você salva 'Pizzaria do João' em vez do número. Sem a agenda, teria que decorar todos os telefones — ou todos os IPs!

REGISTROS E CACHE

Tipos de registros DNS e gestão de cache (TTL)

Tipo	Nome	Função principal	Exemplo de aplicação
A	Address	Mapeia o nome para um IPv4	app.com ➔ 192.0.2.1
AAAA	IPv6 Address	Mapeia o nome para um IPv6	app.com ➔ 2001:db8::1
CNAME	Canonical Name	Aponta um apelido para outro nome	www.app.com ➔ app.com
MX	Mail Exchanger	Define os servidores de e-mail	mail.app.com (priority 10)
TXT	Text	Guarda metadados, SPF e validações	Validação de SSL e DKIM

⏱ TTL — TIME TO LIVE

O que é: tempo, em segundos, que os resolvers podem manter a resposta em cache antes de consultar de novo o autoritativo.

Em migrações: reduza o TTL (de 86400s para 300s) dias antes de trocar o servidor, para propagar rápido e evitar downtime.

RESOLUÇÃO DNS

A hierarquia DNS e a jornada da consulta

1

Cliente e Resolver

O navegador consulta o resolver do provedor ou da nuvem. Se estiver em cache, responde na hora.

→

2

Root Servers (.)

Sem cache, o resolver pergunta aos root servers, que indicam qual servidor de TLD procurar.

→

3

TLD (.com, .br)

O servidor de topo aponta o servidor autoritativo responsável pelo domínio consultado.

→

4

Autoritativo e resposta

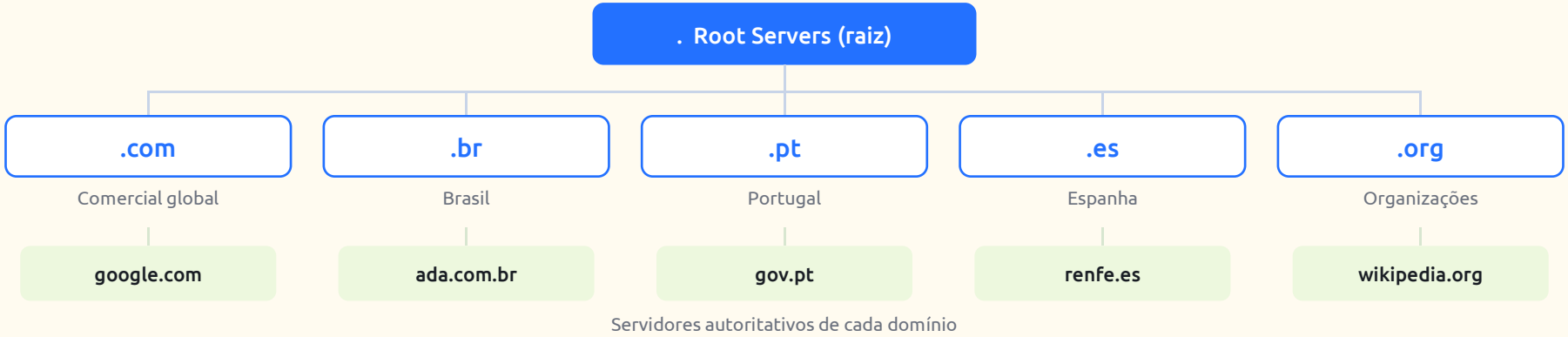
O autoritativo devolve o IP final; o resolver guarda em cache pelo TTL e responde ao cliente.

💡 EM RESUMO

O resolver faz o trabalho pesado: pergunta a três níveis de servidores e guarda o resultado em cache para as próximas consultas.

HIERARQUIA DNS

A árvore do DNS: da raiz aos domínios de cada país



Como o resolver percorre a árvore

A raiz não conhece o site: ela apenas indica quem cuida do .com, do .br, do .pt ou do .es.

O servidor do TLD indica o autoritativo do domínio, que devolve o IP final — sempre da direita para a esquerda do nome.

DNS PRIVADO

DNS Privado: resolução em redes internas (VPC – Virtual Private Cloud)

DNS Público

· Mapeia domínios acessíveis pela internet aberta.

· Acessível por qualquer pessoa no mundo.

· api.suaempresa.com ➔ IP público 200.150.10.5

DNS Privado (Internal DNS)

· Mapeia nomes dentro de uma VPC isolada.

· Não é visível pela internet pública.

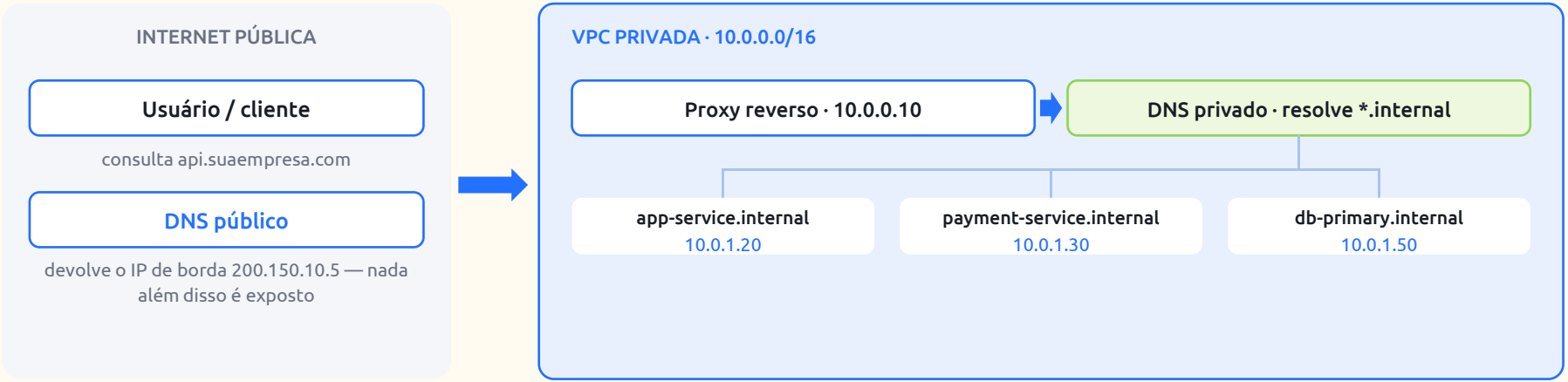
· db-primary.internal ➔ IP privado 10.0.1.50

Por que usamos DNS privado na arquitetura?

1. **Segurança e ocultamento:** Impede o vazamento da topologia da infraestrutura interna.
2. **Desacoplamento de microserviços:** Serviços se falam por nome (payment-service.local), sem depender de IPs.
3. **Ambientes dinâmicos:** Resolução automática para pods e instâncias efêmeras no Kubernetes/Cloud.

DNS PRIVADO NA PRÁTICA

Como a resolução privada funciona dentro da VPC



O que a internet enxerga — e o que fica escondido

De fora, só existe o IP de borda. Dentro da VPC, os serviços se encontram por nome, e o DNS privado devolve os IPs 10.x — que nunca saem da rede interna.

DNS + PROXY REVERSO

DNS e proxy reverso: da borda até a aplicação

Integração na borda

O DNS resolve api.empresa.com e entrega o IP de borda do proxy reverso (Nginx, Traefik, HAProxy).

O proxy recebe a requisição HTTP/HTTPS, encerra o TLS e balanceia para a rede privada dos microserviços.

Roteamento avançado na nuvem (Route 53 / Cloudflare)

1. **Weighted round-robin:** Distribui o tráfego por porcentagem — ideal para testes canary e A/B.
2. **Geoproximity / latency-based:** Direciona o usuário para o data center mais próximo ou mais rápido.
3. **Health checks e failover:** Redireciona o tráfego automaticamente quando o servidor principal cai.

LABORATÓRIO

Comandos de inspeção DNS: Linux e Windows CMD

Objetivo	Comando original (Linux)	Equivalente no Windows CMD
Consulta simples	`dig google.com`	`nslookup google.com`
Registro específico	`dig TXT google.com`	`nslookup -type=TXT google.com`
Rastrear a hierarquia	`dig +trace google.com`	`nslookup -debug google.com`
Consultar DNS específico	`dig @10.0.0.2 api.internal`	`nslookup api.internal 10.0.0.2`
Limpar cache local	`resolvectl flush-caches`	`ipconfig /flushdns`
Filtrar a saída	`dig google.com | grep ANSWER`	`nslookup google.com | findstr Address`
Testar o endpoint HTTP	`curl -I https://app.com`	`curl -I https://app.com`
Rastrear a rota de rede	`traceroute google.com`	`tracert google.com`
Listar arquivos do projeto	`ls -l`	`dir`

Sem equivalente direto: o CMD não faz o +trace do dig — use nslookup -debug para ver a consulta passo a passo, ou instale o dig do BIND. O curl já é nativo no Windows 10 e 11.

RESUMO

Recapitulando a Aula

- ✓ Proxy reverso protege, distribui carga e faz cache — o cliente nunca acessa o servidor direto.
- ✓ DNS converte nomes amigáveis em endereços IP.
- ✓ Registros: A (IPv4), AAAA (IPv6), MX (e-mail), CNAME (apelido), TXT (verificação).
- ✓ Nginx é o proxy reverso mais popular — simples e de alta performance.